VIDYALAYAA / ENGINEERING & OPERATIONS

# A connected school. A dependable release.

## Deployment & operations guide

Version 1.2 | 19 October 2026
Aligned with Software Architecture Specification v1.0

React web application. Express API. PostgreSQL authority.
Independent background worker. NFC attendance services.

Includes environment setup, secrets, database initialization, authentication, hosting, migrations, testing, backups and troubleshooting. External credentials, remaining production firmware source and acceptance gates are identified explicitly.

# 1. Deployment model and boundaries

The React single-page application deploys to Firebase Hosting. A Render web service runs the Express API. A distinct Render background worker drains notification jobs, generates private reports, reconciles stored events and runs school jobs. Supabase PostgreSQL is the only authoritative store. Supabase Auth owns passwords, sessions and MFA. Supabase Storage holds private materials and reports. FCM, transactional email and a selected SMS adapter deliver messages only; they do not hold authoritative attendance.

Use independent staging and production Supabase projects, Firebase projects, Render services and credentials. Choose nearby API/database regions based on the actual provider region inventory. Do not assume a free service provides always-on operation, adequate connections, PITR or a particular recovery window. Size plans from measured load and recovery evidence. The SAS performance figures are targets, not measurements.

Do not deploy a configured school with test students, fake credentials, a public storage bucket, a database owner login in the API, or an unsigned OTA fallback. The setup preview is intentionally disconnected until configured. Real data appears only after authenticated API calls.

# 2. Prerequisites and repository setup

Install a supported Node.js 24 LTS patch, npm, Git, Supabase CLI and Firebase CLI. Docker is needed for the full local Supabase stack. Python with reportlab and pypdf regenerates this guide. PlatformIO builds the firmware example. Chrome or a Playwright Chromium installation runs browser tests.

Create a private source repository and protect its main branch with CI checks. Never add local credentials or the confidential input PDF.

From PowerShell in the source directory:

```
npm ci
Copy-Item .env.example .env
Copy-Item apps/api/.env.example apps/api/.env
Copy-Item apps/worker/.env.example apps/worker/.env
Copy-Item apps/web/.env.example apps/web/.env
npm run build -w @vidyalayaa/contracts
```

All real .env files are ignored. Never add them to version control, a source archive, screenshots or support logs. Only VITE-prefixed public configuration belongs in the browser build. Never prefix a database URL, service-role key, device credential or encryption key with VITE_.

For a browser-only setup preview, leave web configuration absent and run `npm run dev -w @vidyalayaa/web`. To develop real workflows, configure the database and services first, then use `npm run dev` from the root.

# 3. Supabase project and authentication

Create staging and production projects. Save their project URL, public anon key and server-only service-role key in the appropriate secret stores. Restrict project administrator access and enable account MFA. Obtain a direct or session-mode connection for migrations, and transaction-mode pooler connections for runtime roles. Download the correct database TLS root certificate if required by your connection environment; set DB_CA_CERT. Certificate verification remains enabled.

Configure Auth Site URL to the actual web origin. Add only the exact staging/production `/auth/callback` URLs and intended local development callbacks. Disable public sign-up: school administrators invite users. Configure a minimum 12-character password policy, email confirmation, recovery expiry, and leaked-password protection

where supported by the chosen plan. Configure a verified SMTP sender for Auth invitations/recovery; the official support Gmail address is a contact address, not a production bulk sender.

Enable TOTP MFA enrollment/verification. Super Admin routes enforce verified aal2 tokens. School administrators and principals should enroll MFA; apply the school's operational policy. Keep emergency recovery identities in a controlled password manager and test recovery without weakening MFA.

The migration creates an auth.users trigger that maintains the application profile. If identities predate the migration, a migration owner must explicitly backfill their profiles from verified Auth identities. Do not infer identity from a client-supplied role or user metadata. Role assignment comes from database memberships only.

# 4. Database initialization and runtime roles

Set MIGRATION_DATABASE_URL in the root environment to the migration-owner direct/session connection. Set DB_SSL=true and the root CA where needed. Apply the authoritative forward migrations:

```
npm run db:migrate
npm run db:verify
```

The migration runner takes an advisory migration lock, applies each SQL file transactionally, and records its checksum in app.schema_migrations. A changed checksum for an applied file stops deployment. Do not edit an already applied migration. SQL lives in db/migrations; generator scripts are maintenance aids, not a deployment step.

This release has nine ordered migrations. Upgrading an existing five-migration installation applies 006 school invitations and first-admin bootstrap, 007 invitation activation guards, 008 worker operations/push targets/provider quotas/device snapshots, and 009 opt-in retention. Run the same migration and verification commands before deploying the matching API and worker. Migration 009 does not enable cleanup for any school: a School Admin must approve a policy first. Verify the checksum ledger, forced RLS, role grants and invitation guards with db:verify and the database tests; verify both empty-database and five-to-nine upgrade paths in staging. An old API image predating invitation acceptance must not manage memberships after this upgrade; keep the compatible image or disable those writes during rollback.

The migrations create non-login, non-superuser, non-BYPASSRLS group roles app_api, app_device, app_worker and app_platform. Provision distinct login accounts with unique random passwords. Place the four DB_API_PASSWORD, DB_DEVICE_PASSWORD, DB_WORKER_PASSWORD and DB_PLATFORM_PASSWORD values in your local provisioning environment or approved secret injection process, then:

```
npx tsx scripts/provision-db-roles.ts
```

The script never prints passwords. It creates vidyalayaa_api, vidyalayaa_device, vidyalayaa_worker and vidyalayaa_platform as ordinary inheriting logins and grants exactly their matching group. Use the runtime login-specific pooler URL format shown by your Supabase dashboard. Test that the project/role suffix and pooler mode are correct; do not guess a pooler hostname.

Runtime mappings: API DATABASE_URL uses vidyalayaa_api; DEVICE_DATABASE_URL uses vidyalayaa_device; PLATFORM_DATABASE_URL uses vidyalayaa_platform; worker WORKER_DATABASE_URL uses vidyalayaa_worker. The migration-owner URL is never provided to either service. Runtime startup validates that each connection uses its expected role, owns no application tables and has no superuser, role-creation or BYPASSRLS capability; production also refuses disabled database TLS. Service-role keys are used only server-side for Auth administration and authorized Storage operations, not for school SQL queries.

All school queries execute within a transaction that calls transaction-local set_config for school, actor and request. RLS is enabled and forced. Composite foreign keys prevent school A children pointing at school B parents. No browser anon/authenticated PostgREST privilege is granted on application tables. app_platform cannot select students, attendance, fees or marks. Audit logs allow application INSERT/SELECT only; raw tap fields and rule contents are guarded against modification.

The production seed migration contains only system roles, permission mappings and configurable plan defaults. Plan limits are initial development defaults, not contractual entitlements. Review and set the actual school limits before onboarding. The optional development seed creates two labelled development schools without real children or credentials and is refused unless ALLOW_DEVELOPMENT_SEED=true and NODE_ENV is not production.

# 5. Local Supabase and isolated tests

With Docker running:

```
supabase start
```

Use the local URLs and public/server keys returned by your own CLI. Point MIGRATION_DATABASE_URL to the local database owner, set DB_SSL=false only for localhost, and run the migration/role commands above. Runtime accounts still use separate roles locally. Configure local Auth callbacks and invitation delivery via the local mail testing interface. `supabase db reset` destroys local data; it is not a production migration command and does not replace the repository's migration runner.

Portable tests need no Docker and execute PostgreSQL in WASM. They validate SQL, RLS, foreign keys, API scope and workflow transactions. That environment has one backend; it does not replace the multi-connection row-lock test. CI starts PostgreSQL 17 and applies the exact migrations using isolated Auth/Storage metadata fixtures.

For an already created local database named vidyalayaa_test, configure TEST_DATABASE_URL and MIGRATION_DATABASE_URL to it, then:

```
npx tsx scripts/prepare-test-db.ts
npm run db:migrate
npm run db:verify
npm run test:integration
```

The preparation script refuses a database name not ending in _test. Never aim it at a real Supabase project. CONCURRENCY_ITERATIONS defaults to 1000, exercising five readers, same-event retries and a single daily/outbox result. Fault injection, pooler failover, hardware and restore drills still require dedicated staging environments.

# 6. First platform and school administrators

Create the first identity through Supabase Auth's controlled administrator interface, verify it, and record its actual UUID. The Auth trigger creates its profile. Link this specific identity:

```
npx tsx scripts/onboard.ts platform VERIFIED_AUTH_USER_UUID
```

Sign in, enroll and verify the authenticator in Settings, then use the platform console. Register a school with its unique code, name, IANA timezone, plan and trial end. Add academic sessions, classes, subjects and an approved rule version in the school workspace. Rule examples are never installed automatically as a school's approved policy.

In the Super Admin console, open Schools and choose Invite School Admin on the unstaffed school's row. Enter the administrator's name and email. The platform can bootstrap only this first School Admin; routine membership management belongs to the school. An already staffed school shows School Admin assigned. Reopening a pending invitation provides resend and cancellation controls. Sending a setup email is a real external action, so use an intended recipient rather than a live test account.

For a new identity, Supabase sends a setup link. The recipient opens it at /auth/callback, verifies the address and chooses a separate Vidyalayaa password. The page verifies the provider session before changing the password, supports invite/recovery token fragments and PKCE codes, and removes sensitive callback parameters from browser history. Keep the local web server running when using a localhost callback. Expired/used links require a new invitation.

The recipient must accept the school invitation before its INVITED membership becomes ACTIVE. An existing verified identity receives a pending invitation in Settings and can accept it with its current password; no duplicate Auth account or membership script is needed. The same person may belong to multiple schools with different roles. Invitations default to seven days; mail dispatch has a two-minute retry lease. Provider failures remain visible and can be retried. Cancelling or declining prevents acceptance. A database trigger prevents activating an unaccepted invitation through a status detour.

The School Admin then opens Members & invitations, chooses Invite member, and selects Principal, Teacher, Parent, Student or another School Admin. All users sign in at /login with email/password or Continue with Google. The application routes School Admins to /admin, Principals to /principal, teachers to /teacher, parents to /parent and students to /student. The Super Admin starts at /platform; platform subpages have direct URLs such as /platform/schools and /platform/health. Permissions and row scopes remain server-enforced; typing a different portal URL does not change access.

For Google login, create a Web application OAuth client in Google Auth Platform with only openid, email and profile scopes. Choose the External audience for multiple schools and add test users while the Google app is in Testing. Set the Google client's authorized redirect URI to the exact Supabase Auth callback shown in the Google provider settings, normally https://YOUR_PROJECT_REF.supabase.co/auth/v1/callback. Enable Google in Supabase Authentication and save its Client ID and Client Secret there. Never put that secret into a browser environment variable.

In Supabase URL Configuration, allow the website callbacks http://localhost:5173/auth/google/callback and http://127.0.0.1:5173/auth/google/callback for local development, plus the actual deployed HTTPS origin followed by /auth/google/callback. Preserve the separate /auth/callback entries for invitations/recovery. Keep the same browser and origin throughout the PKCE redirect. Enable Auth signups if unregistered Google identities may sign in; they receive no school access until accepting an invitation. The local Supabase template defaults to invitation-only identity creation.

Both login methods prompt for an enrolled authenticator. The API rejects authenticated data requests below aal2 when the verified identity has an enrolled factor; platform endpoints always require aal2. No Google email/profile claim assigns privileges. Supabase can link identities with the same verified email, preserving the existing account's membership. Full setup, provider error troubleshooting and acceptance commands are in docs/GOOGLE_SIGN_IN.md. A real Google login remains a deployment acceptance step until its external credentials are configured.

After acceptance, create the relevant teacher/parent/student profile using the named account selector. Assign teachers to classes/subjects, link parents to students, and enroll students in the correct session/class. Invitation acceptance alone does not create a teaching assignment or parent-child link. The last active School Admin cannot be removed or demoted until another administrator is active. The owner-only onboarding script remains a controlled bootstrap/recovery tool, not the normal website workflow.

# 7. Web application and Firebase Hosting

Set the three required public web values: VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY and VITE_API_URL. Use the staging versions for staging. Vite bakes them into static files at build time; changing Render variables cannot change an already built Firebase app. Fabric Grotesk is used when locally available; the fallback is Arial/system sans-serif. If distributing a licensed web font, add the approved WOFF2 asset and its @font-face URL before building.

Copy .firebaserc.example to .firebaserc or pass --project explicitly. Review firebase.json CSP connect-src against the exact API/Auth origins. The default permits Supabase and Render provider domains; add a custom API domain explicitly when using one. Keep script-src self, no framing, nosniff, HSTS and the SPA rewrite. Static assets are immutable; index.html is not cached indefinitely.

```
npm run build
firebase login
firebase deploy --only hosting --project YOUR_STAGING_FIREBASE_PROJECT
```

After staging acceptance, build with production public values and deploy to the production Firebase project. Use a least-privilege service account in GitHub Actions rather than a personal interactive login. Verify deep links such as /attendance load directly, login callbacks return to the intended origin, and no service-role key appears in the built bundle.

# 8. Render API and background worker

Import render.yaml from your GitHub repository. Choose always-on service capacity and a region appropriate for your database/schools. The API and worker are separate Docker processes with independent deployments. Both build from the monorepo Dockerfile; the runtime user is non-root. Auto-deploy is off so reviewed migration/release ordering controls rollout.

Configure every required API secret from apps/api/.env.example. WEB_ORIGINS must contain exact allowed web origins, comma-separated, without wildcard origin authorization. Configure the known proxy hop count; render.yaml starts with one Render proxy hop, which must be verified for custom routing. Do not set trust proxy=true without a defined topology. DB_SSL must remain true for managed database connections.

Configure the worker from its separate example. TOKEN_ENCRYPTION_KEY is the same random 32-byte key, encoded as 64 hexadecimal characters, in API and worker. Keep DB_POOL_SIZE small initially (API 10, worker 5) and reserve measured headroom for devices, platform requests, maintenance and pooler limits. Every API instance creates separate API/device/platform pools, so account for all pools when sizing connections.

Local container commands:

```
docker build -t vidyalayaa:local .
docker run --rm --env-file apps/api/.env -p 4000:4000 vidyalayaa:local
docker run --rm --env-file apps/worker/.env vidyalayaa:local node
  apps/worker/dist/apps/worker/src/index.js
```

Use the exact entrypoints above on Render. API health checks target /health/ready; /health/live only confirms the process. The worker exposes no public HTTP port. Its private worker_instances heartbeat records liveness, last successful cycle and a safe error code. Platform Health displays this metadata. After building, private command-line checks are:

```
npm run health:live -w @vidyalayaa/worker
npm run health:ready -w @vidyalayaa/worker
```

Liveness requires a heartbeat within 90 seconds; readiness also requires RUNNING state and a successful cycle within five minutes. These are fleet checks over the private worker connection. Set a distinct WORKER_INSTANCE_ID UUID only when assigning stable identities to individual instances. Ensure Render's deployed commit matches the approved release; an old healthy instance is not evidence that a new deploy finished. Verify the worker's actual Render deployment status and recent completed job_runs after rollout. Configure log drains and on-call alerts outside the application source.

# 9. Storage and private reports

The migrations create private school-documents and firmware buckets. school-documents is restricted to PDF, PNG and JPEG, up to 10 MiB. The browser cannot list storage objects. Material upload URLs are issued only after class/subject authorization and point to a randomized path under the authorized school. Do not loosen storage policies to resolve a failed upload.

Reports queue in report_documents and are generated by the worker under current membership and row scope. The worker writes a PDF to school_id/reports/report_uuid.pdf. The browser receives a 60-second application-signed URL bound to user, school and report; fetching it also requires the same valid JWT and current scope. The server streams the private object. Copying a link to another account does not authorize it. Large reports above 10,000 selected rows fail explicitly; request a narrower period/scope.

Academic reports include per-student/per-exam totals, weighted percentages and subject remarks. Their period uses the examination date, with school-timezone mark creation date only when the exam has no date. Family exports contain only published results. Missing marks are not silently treated as zero and no grading rubric is invented.

Back up object storage separately from PostgreSQL. Database backups do not imply that object bytes are recoverable. Automatic report/notification cleanup is inactive until a School Admin saves an approved policy reference in Settings. Report age is capped by the effective plan report_retention_days; completed notification history uses the approved notification period. Cleanup expires generated PDF metadata only after Storage deletion succeeds and purges completed delivery/attempt/outbox records in bounded batches. Pending and dead work, student records, fee records, raw attendance and append-only audit history remain. Stop future cleanup removes the policy for the next worker cycle; already removed content needs backup restoration. Orphaned materials and legal child-data deletion remain separate reviewed operations.

# 10. Notifications and providers

Push: configure a Firebase project and service-account JSON on the worker as FIREBASE_SERVICE_ACCOUNT_BASE64. Configure the public web Firebase identifiers and VAPID key, rebuild the web app, and enable notifications from a parent account in Settings. The first-party service worker displays school updates without embedding a Firebase server key. Tokens are encrypted with authenticated AES-GCM and indexed by a hash. Logout deactivates the current browser token; invalid/inactive tokens are retired by the worker.

Email: the supplied Resend adapter requires RESEND_API_KEY and a verified EMAIL_FROM domain. Configure SPF, DKIM and DMARC and test deliverability to representative parent providers. Auth invitation SMTP is a separate Supabase Auth setting. Do not use the official support Gmail account as a bulk transactional sender.

SMS: select a provider and implement/configure an HTTPS adapter accepting `{to,message,template,idempotency_key}` and returning `{message_id}`. Set SMS_ADAPTER_URL, SMS_ADAPTER_TOKEN and SMS_TEMPLATE_ID. Verify current sender/template/operator registration

requirements with that provider before enabling real sending. No real SMS provider is claimed by this source until selected and tested.

Unconfigured adapters fail visibly with PROVIDER_UNCONFIGURED; they never report a fake successful delivery. Preferences default push/email on and SMS off, with independent records for every linked parent and channel. Quiet hours, stale messages, plan gating and daily caps are applied. Each push token has an independent durable target result: an invalid token is disabled individually, other devices still receive the message, and successful targets are skipped on a transient retry. A provider acceptance followed by a crash can still duplicate a message. Shared database token buckets throttle provider calls across worker instances; configure PUSH/EMAIL/SMS_RATE_PER_SECOND and corresponding _BURST values for actual provider quotas. Recipient-specific permanent errors do not trip the provider outage circuit. Review DEAD jobs and provider errors daily.

School profile notification locale can be en-IN or hi-IN. Templates are keyed by notification kind and locale; current messages omit child names from lock-screen text. Provider sender/template registration must match the configured language and approved message content.

External delivery is at least once. Resend receives a stable idempotency key. Push/SMS providers may deliver twice if the process dies after provider acceptance but before database recording; a database transaction cannot make an external provider exactly once. Confirm provider idempotency or client-side dedupe where required. Never tell a parent that a provider acceptance ID proves the notification was read.

# 11. NFC hardware provisioning and rollout

Platform staff register each physical device to one school. Registration returns its random secret and claim code once; store them in a controlled provisioning system. Only their SHA-256 hashes persist server-side. The School Admin claims it into an active ENTRY, EXIT, BIDIRECTIONAL or CLASSROOM location. Devices have no authority to choose school/student IDs.

Review docs/DEVICE_API.md with the embedded team. Build the PlatformIO example, initialize an empty queue only on a new unused device, inject only that unit's credential and real API root CA, then flash it. Green feedback means the raw tap was durably stored locally, not that the server classified attendance. Block/replace lost cards using lifecycle records; do not reuse an active UID across students.

The selected family is ESP32 Dev Module with Wi-Fi and BLE (PlatformIO esp32dev). No board is connected yet, so chip revision and actual flash capacity remain unverified. Inspect those read-only when the board is connected, record the module marking and partition capacity, and approve a matching secure production layout before flashing or setting irreversible security fuses. The current example must not be treated as that production image.

The included firmware is a prototype example with explicit production limitations in firmware/README.md. Its blocking network upload and file-per-event queue must be replaced/validated for a busy gate. Production units require secure boot, flash encryption, encrypted NVS, signed A/B OTA, anti-rollback, watchdogs, queue capacity/wear/power-loss evidence and the actual board's security configuration. API OTA status inventory exists, but remote installation is disabled until signed production firmware is implemented and tested. Never enable unsigned updates to meet a rollout date.

Validate five simultaneous readers, retry after a lost response, 24-hour backlog, reboot with pending events, RTC power loss/drift, storage full/corruption, credential revocation and Wi-Fi coverage. Preserve pending events during revocation or replacement. Subscription expiry does not gate event ingestion for non-revoked devices. Reconcile a replaced/clock-damaged unit's queue rather than silently discarding it.

# 12. Fees, subscriptions and billing

Fee records track charges, partial/full payment and waivers with numeric constraints and audit history. They do not charge a bank account. The SAS leaves the online billing provider open. Choose a provider and verify its native signature in a trusted adapter before transforming to the documented signed metadata webhook. Do not point an unverified payment browser redirect at subscription state changes.

Set BILLING_WEBHOOK_SECRET only when that adapter exists. The API verifies a timestamped HMAC, rejects stale/replayed altered events and updates only subscription metadata through a narrow database function. Test duplicate provider events, retries and invalid signatures in staging. Keep prices/invoices with the selected billing provider; the source does not invent prices.

Lifecycle defaults follow the SAS proposal: expired ACTIVE/TRIALING becomes PAST_DUE, then SUSPENDED_READONLY after the contractual grace (default 15 days), then SUSPENDED after 30 more days. School administrators retain restricted billing/export access. Data is not deleted. Alerts can pause while attendance ingestion continues. Set contractual grace/limits explicitly, and review subscription_events and audit history for every change.

# 13. CI/CD and release procedure

Platform operations in this release: edit plan feature flags and numeric limits in Plans, with a reason recorded in append-only audit history. Subscription edits have named controls and explicit school overrides; omitted API override/grace fields are preserved. Health shows aggregate usage, queue gaps, quarantines, notification/report backlog and private worker heartbeat state. No student directory is exposed to the platform role.

To inspect school records for support, request only the needed scope and ticket duration. The school approves the request in Support access. Open approved access from the platform grant row; the read-only view hides data after expiry or a failed/revoked refresh. The server revalidates and locks the live grant in the same transaction that reads and audits each returned record. A school notice lists active support access and provides revocation controls. General membership management does not become available through a support grant.

School resource details are readable by authorized roles even when they cannot edit. Homework materials download through fresh authorized URLs. Account/student/class selectors use names and scoped server searches. Attendance has class/date filtering, pagination and correction buttons on each daily record. Resource drafts are held in memory for up to one hour, scoped to user/school, cleared on logout/account or school change, and not written to browser storage. Dates are displayed and entered in the school timezone; ambiguous or nonexistent daylight-saving times are rejected. Notification/audit feeds support cursor pagination so new records do not shift a later page. Losing network connectivity displays an offline notice; it does not queue school-data writes in browser storage.

If an API session expires, the client tries one forced refresh. On failure it keeps an inaccessible in-memory draft checkpoint while returning to sign-in. Signing back in as the same identity restores the draft and selected school/return path; a different identity or explicit sign-out discards it. Reloading the browser discards all memory-only drafts. MFA sign-in also preserves the return path. Do not use browser storage as a workaround for sensitive draft recovery.

ci.yml runs TypeScript checks, local PostgreSQL/WASM API tests, schema assertions, real PostgreSQL five-device concurrency tests, generated-contract drift checks, production builds, secret scans and browser/accessibility tests. Require these checks on pull requests. Configure GitHub environments named staging and production; production requires a named reviewer and protected branch/tag policy.

Run the local source checks before releasing:

```
npm run typecheck
npm test
npm run build
npm run contracts
npm run check:secrets
npm audit --omit=dev --audit-level=high
npx playwright test tests/e2e/workspace.spec.ts tests/e2e/mfa.spec.ts
  tests/e2e/auth-callback.spec.ts tests/e2e/google-login.spec.ts
  tests/e2e/onboarding.spec.ts tests/e2e/school-workflows.spec.ts
  tests/e2e/platform.spec.ts --workers=2
```

These browser suites launch isolated fixture servers, ignore local service credentials and never send real invitations. They exercise desktop/mobile role entry, invitations, MFA, password setup, plans, expiring support, record drafts, attendance and private homework access. The separate live.spec.ts release test needs E2E_BASE_URL, E2E_EMAIL and E2E_PASSWORD for the intended staging account; set REQUIRE_LIVE_E2E=true to make missing credentials fail the release. See output/verification.json for the checks actually executed in this delivery. Fixture tests do not establish production provider or hardware acceptance.

Configure environment-scoped secrets: MIGRATION_DATABASE_URL, RENDER_API_DEPLOY_HOOK, RENDER_WORKER_DEPLOY_HOOK, FIREBASE_SERVICE_ACCOUNT_JSON, and staging E2E_EMAIL/E2E_PASSWORD. Set environment variables VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_API_URL, FIREBASE_PROJECT_ID and STAGING_WEB_URL. Provider/database runtime secrets stay in Render, not the browser or GitHub public variables.

Dispatch Deploy reviewed release for staging at the intended commit. Apply additive migrations, deploy API/worker, verify the deployed commit and worker health, then publish the matching frontend and run authenticated staging smoke. Production dispatch requires a non-expired staging-pass artifact for the identical commit. API rollout waits until readiness reports the expected Render commit. Verify the independent worker deployment in Render before promotion. Promote the identical reviewed commit only after the staging gate succeeds. A successful hosting upload alone is not full system readiness. Retain migration logs, test artifacts, service deploy IDs and the approved readiness checklist.

# 14. Migration safety and rollback

Use expand/contract: add compatible columns/tables/indexes, deploy code that tolerates old/new shapes, backfill under observation, verify invariants, then remove old fields in a later approved migration. Take an on-demand backup/snapshot before risky changes and verify available restore points. Test empty-db and upgrade migrations in staging.

If a release fails, keep already committed attendance and outbox data. Roll back API/worker to the previous compatible Render image and Firebase to the previous Hosting release. Do not reverse a migration by dropping attendance or audit tables. Applied checksums remain immutable. Repair schema with a new forward migration. If a destructive migration truly needs recovery, use the restore procedure into a new project, validate it, and perform an explicit controlled cutover.

Database transactions roll back an event, projection and outbox together on failure. The device sees RETRY_LATER or a connection failure and retains the same UUID for retry. Never manually acknowledge an event before its commit to clear a queue.

# 15. Backups, restoration and disaster recovery

Enable the actual provider's daily backups and PITR where contracted. SAS targets RPO <=5 minutes and RTO <=4 hours, which require validated PITR/recovery capability; a daily backup alone cannot meet a five-minute RPO. Restrict restore access to named engineers, enable administrative MFA, encrypt exported backups and keep access audited. Back up storage objects and configuration/secret recovery separately.

Quarterly, restore the chosen point into an isolated scratch project. Record source/restore timestamps and duration. Apply missing forward migrations; run schema verification, row-count/constraint checks, cross-school tests, Auth/membership checks and a non-production device retry simulation. Check report/material bytes exist in storage and match the restored database paths. Do not send real parent notifications from the scratch worker.

During an incident: keep devices queueing; stop automated external notification sending if restored state is uncertain; preserve provider idempotency records; determine the correct recovery point; restore; validate; update API/worker connections and keys; then resume controlled device upload and reconcile late taps. Rotate compromised credentials independently of database restoration. Record measured data-loss and downtime against the targets. A restore drill must finish before claiming production readiness.

# 16. Monitoring, alerts and troubleshooting

Monitor API readiness/error rate/latency, pool use, lock waits, disk growth, pending/dead outbox age/count, provider failures, report failures, worker job freshness, device last-seen/queue/RTC/reader/storage and backup age. Initial SAS alert suggestions include offline devices after three missed heartbeats, oldest outbox >5 minutes and any DEAD notification; tune them using pilot evidence. Configure on-call delivery and test the alerts.

Logs use request IDs and stable error codes; tokens, full card UIDs and child data are not included in request logs. Application audit logs contain authorized business-change evidence and must be access controlled. Query plans should be reviewed on realistic staging volumes: school/date attendance register, student-day replay, parent fanout, teacher class scope and outbox claims. School/foreign-key indexes support these query paths; remove redundant indexes only after EXPLAIN evidence.

Sign-in loop: verify public project URL/key, Auth callback allowlist, account status and membership. A valid Auth account without an active membership is intentionally unable to view school data. Platform 403 MFA_REQUIRED means complete the verified authenticator challenge; never disable the middleware.

Broken MFA QR: the UI renders Supabase's SVG data URI directly, with a manual secret only during active enrollment. Reopening setup rechecks factors and restarts only the app's unverified stale enrollment; it preserves verified factors and unrelated authenticators. If cleanup fails, fix the provider error and retry rather than deleting verified factors. A localhost connection-refused invitation page means the development web server is stopped; start it and request a fresh setup link if the old link was consumed or expired. Keep production Auth callbacks on the deployed HTTPS origin.

Database connection failure: verify runtime login, role membership, pooler project suffix/mode, TLS CA, provider network rules and total connections. Do not use the migration owner as a workaround. RLS returning no rows: check the verified membership, X-School-Id selection and transaction-local context; never disable RLS.

No attendance: inspect device health, card ACTIVE status, enrollment date/class, approved rule effective range, school calendar and timestamp trust. Pending rules or quarantined timestamps remain stored for reconciliation. A missing first arrival with an EXIT event is flagged; uploading the earlier arrival replays the day. After a manual correction, the override remains authoritative.

No notifications: inspect outbox/delivery status, plan state, linked active parents, can_receive_alerts, preferences/quiet hours, token age, provider configuration and staleness. A saved tap with a failed provider is

expected durable behavior. Restart-safe leases recover jobs; do not delete the outbox to make a dashboard look clean.

Report pending: verify the separate worker process and job freshness. Report failed: check current membership/scope, maximum row count, private bucket and service key. Download expired: request a fresh link while signed in to the same school. Never make a bucket public as a fix.

Browser blocked by CSP/CORS: add only the verified intended API/Auth origin and rebuild/redeploy the appropriate host. Blank deep link: verify Firebase's SPA rewrite and immutable asset/index cache separation. Device TLS failure: check real root CA, system time and certificate chain, never setInsecure.

# 17. Final pilot acceptance

Review docs/READINESS.md and the SAS Section 30 checklist. Close all critical/high security findings, run live multi-school/role journeys and provider tests, complete hardware power-loss/OTA and card-cloning mitigation review, obtain legal/retention/consent sign-off, measure load and connection headroom, perform a restore drill, validate Wi-Fi/site power and train school staff. Record the named owners and evidence. Shipping source and a passing build do not by themselves satisfy these external gates.

Record a named owner, evidence link and acceptance decision for each gate:

- Identity and isolation: cross-school/IDOR tests, parent and teacher scope, MFA and credential revocation.
- Attendance: simultaneous readers, retry after lost response, rule boundaries, replay, corrections and absence retraction.
- Hardware: signed secure firmware, 72-hour durable queue, power-loss/RTC/OTA rig, Wi-Fi survey and spare units.
- Notifications: verified providers, every linked parent, preferences, retries/dead letters, stale alerts and on-call routing.
- Recovery: database PITR and object backups, isolated restore drill, measured RPO/RTO and key recovery.
- Release: approved commit, staging authenticated tests, migration compatibility and independent API/worker/hosting checks.
- Privacy: lawful processing, consent/notice, contractual retention, support-grant approval and closed security findings.
- Operations: measured load and connection headroom, trained school staff, monitoring ownership and incident drill.

Primary references: PostgreSQL row security (https://www.postgresql.org/docs/current/ddl-rowsecurity.html), Supabase identity verification (https://supabase.com/docs/reference/javascript/auth-getuser), Firebase Hosting (https://firebase.google.com/docs/hosting/full-config), Render Blueprints (https://render.com/docs/blueprint-spec), and Resend idempotency (https://resend.com/changelog/idempotency-keys). Vendor controls and plan capabilities must be checked against the selected account at deployment time.